ISSN 1009-3044
Computer Knowledge and Technology 电脑知识与技术
Vol.13, No.22, August 2017

E-mail:kfyj@dnzs.net.cn
http://www.dnzs.net.cn
Tel:+86-551-65690963 65690964

基于Python和SIPp的自动化压力测试系统设计和实现

信斌[1],王桂花[2]

(1.南京中兴软件有限责任公司,江苏 南京 210012;2.江苏经贸职业技术学院,江苏 南京 211168)

摘要:首先,分析了现有测试系统的不足,之后,介绍了软件组成和运行环境说明,最后,设计了基于Python和SIPp的自动化压力测试系统,并给出了相关代码。新的测试系统克服了现有测试系统的不足,极大地提高了测试效率,节约了测试成本。

关键词:压力测试系统;自动化;Python;SIPp

中图分类号:TP311 文献标识码:A 文章编号:1009-3044(2017)22-0205-03

DOI:10.14004/j.cnki.ckt.2017.2563

Design and Implementation of Automatic Pressure Testing System Based on Python and SIPp

XIN Bin[1], WANG Gui-hua[2]

(1.ZTE Corporation, Nanjing 210012,China; 2.Jiangsu Vocational Institute of Commerce, Nanjing 211168, China)

Abstract: Firstly, analyze the shortcomings of the existing test system. Secondly, introduce the software composition and operating environment. finally, design the automatic pressure testing system Based on Python and SIPp, and give the relevant code. The new testing system overcomes the shortcomings of the existing testing system, greatly improves the testing efficiency and saves the test cost.

Key words: pressure testing system; automatic; Python; SIPp

随着计算机和通讯技术的快速发展,网络和通讯设备的组成越来越复杂,测试工作量越来越大,测试要求的时间越来越短,测试系统面临着很大的挑战。现有测试系统中,基于SIP协议的通讯系统的压力测试,多采用开源工具SIPp,由测试人员手工调用SIPp执行脚本,给系统加压,然后采集系统运行指标,如CPU占用率、内存使用率等,以测试系统在不同压力下的稳定性。SIPp工具模拟终端设备或者网络设备,向被测试设备发送大量SIP消息,可以增加系统负荷。但由于压力测试的测试模型众多,测试周期较长(通常持续若干天),测试成本居高不下。

Python是一种开源的通用计算机编程语言,致力于缩短开发时间,使用简洁、易读又便于维护的语法[1],拥有丰富的扩展模块,具有很大的便捷性和高可扩展性[2]。基于Python和SIPp的自动化压力测试系统,将人工测试步骤编排为自动化测试,能极大地提高测试效率,节约测试成本。

1 系统组成

1.1 系统的硬件结构

自动化压力测试系统由测试管理子系统、SIPp测试节点和被测试设备组成,系统硬件连接如图1所示。根据测试规模和被测试设备的能力,可以扩展多个SIPp测试节点,由测试管理系统集中管理和分配任务。所有设备通过交换机和路由器组成的网络连接在一起。测试管理系统使用网络接口,一方面管理SIPp节点,启动及停止其模拟负荷,并在测试过程中动态调整其负荷大小,另一方面监控被测试设备,判断其在高压力情况下的运行指标及状态。

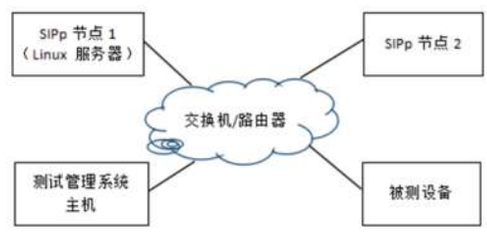

图1 系统硬件连接示意图

1.2 系统的软件结构

本系统由配置管理模块、SSH协议接口模块、SIPp控制模块、设备参数采集模块、测试报告生成模块组成,具体结构见图2所示。

配置管理模块负责读取测试系统的各项预设参数(如被测试设备的IP地址、端口、登录口令,SIPp节点的Linux服务器用户名、口令、测试消息脚本等)。

因SIPp工具运行于Linux系统上,需要通过SSH协议登录到Linux服务器,以便控制SIPp的启动、停止等动作,工具需要支持SSH接口。将SSH的连接、登录、执行命令、获取命令结果

收稿日期:2017-06-18

作者简介:信斌(1973—),男,河南商丘人,工程师,学士,研究方向为软件测试;王桂花(1976—),女,山东沂水人,副教授,管理学博士,研究方向为系统管理及优化。

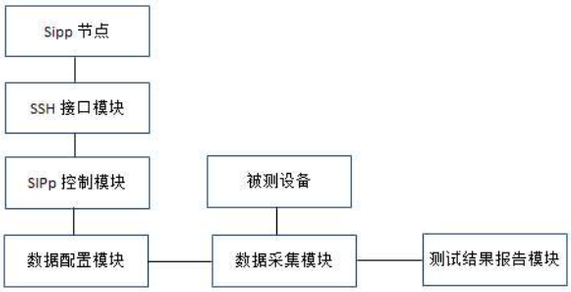

图2 系统软件模块结构图

等操作封装为一个SSH模块，方便工具其他模块使用。SIPp控制模块负责在部署SIPp工具的Linux服务器上，启动测试脚本、调整压力强度及结束模拟负荷的发送。数据采集模块通过TCP接口登录到被测试设备上，采集各种强度负荷下的设备系统指标，判断系统的状态是否良好。测试结束后，所有测试数据由报告生成模块加工生成测试报告。

2 运行环境说明

2.1 Python扩展模块paramiko的安装

paramiko模块提供了ssh及sftp进行远程登录服务器执行命令和上传下载文件的功能。这是一个第三方的Python扩展包，使用之前需要安装。在Windows上安装paramiko包的步骤具体见表1所示。

表1 paramiko安装步骤

顺序	安装过程
1	下载安装Windows版本的Python2.7
2	下载安装PyCrypto2.6 for Python 2.7 64bit
3	下载安装ecdsa-0.10.tar.gz
4	下载安装paramiko.tar.gz
5	确认安装成功，进入python环境，输入import paramiko无错误提示则表示安装成功!

2.2 SIPp工具的安装与测试脚本编写

SIPp是一款用于测试SIP协议性能的开源工具软件，运行于Linux平台，通过预先编排的测试场景脚本来控制SIP消息的收发，它可以模拟用户终端以及通讯服务器设备，完成和被测试设备的SIP协议流程的交互。使用不同的脚本配置，它可以充当SIP协议中的UAC角色和UAS角色，并发或者串行地发起多个SIP会话，并能实时输出测试运行的统计数据(呼叫速率、信令来回的延迟，以及 消息收发数目统计)[3]。由于其性能强大和开源免费的特点，SIPp已经被广泛应用于通讯设备(如IP多媒体子系统中的CSCF、AS等设备)的压力测试，具体步骤如图3所示。

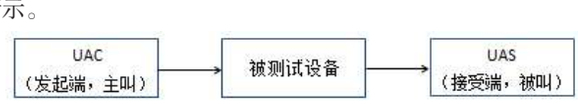

图3 SIPp进行AS的压力测试的步骤

其中UAC和UAS都由SIPp来担任。SIPp的安装比较简便，从开源网站上下载代码，拷贝到Linux服务器上，完成编译即可。但在测试前，需要完成测试场景脚本文件(XML格式)和用户号码脚本文件(CSV格式)的设计与编写，并将消息收发流程调试通过。

3 系统模块功能设计与实现

3.1 SSH接口模块的设计与实现

对第三方模块paramiko的接口进行二次封装，将繁琐的协议函数封装为简单的接口，主要包括执行命令接口、上传文件接口及下载文件接口。

SSH接口模块分为几个部分：

3.1.1 SSH初始化接口

首先导入paramiko模块

```
import paramiko
```

然后定义SSH封装类及其初始化函数：

```
class ssh_module():
def __init__(self, hostname,port,username,passwd):
self.ssh = paramiko.SSHClient()
self.ssh.load_system_host_keys()
self.ssh.set_missing_host_key_policy(paramiko.AutoAddPoli -
cy())
self.sftp=None
self.channel=None
self.ssh.connect(hostname,port,username,passwd,timeout=2)
self.sftp = self.ssh.open_sftp()
self.channel = self.ssh.get_transport().open_session()
self.channel.get_pty()
self.channel.invoke_shell()
self.channel.settimeout(20.0)
self.connect_status=1
```

3.1.2 SSH命令执行接口：

```
def ssh_exec_cmd(self,cmd):
stdin,stdout,stderr=self.ssh.exec_command(cmd)
out=stdout.read()
err=stderr.read()
return out,err
```

3.1.3 SFTP文件传递操作接口

```
def upload_file_to_remote(self,localpathfile,remotepathfile):
self.sftp.put(localpathfile,remotepathfile)
return
def download_file_from_remote(self,remotepathfile,localpath-
file):
self.sftp.get(remotepathfile,localpathfile)
return
```

3.2 SIPp控制模块

SIPp工具本身运行于Linux服务器上，需要调用ssh接口，远程执行SIPp的运行脚本，从而控制SIPp向被测试设备发起SIPp消息，模拟加载负荷。 SIPp控制模块有如下几个部分：

3.2.1 启动SIPp脚本

需要预先在运行SIPp工具的linux服务器上，安装好SIPp程序，编排好sip消息收发流程脚本文件(xml格式)和用户号码本脚本文件(csv格式)，然后以linux shell命令的方式启动SIPp：

```
SIPp_cmd="/TestTools/SIPp 10.5.1.20：5060 -sf /TestTools/
```

reg.xml -inf /TestTools/Reg_user.csv -i 10.5.6.80 -p 5501 -r 100 -m 200000 -bg”

以上代码定义了一个字符串变量SIPp_cmd，其内容是一个SIPp启动命令，该命令执行reg.xml中定义的消息收发序列，并以Reg_user.csv中定义的用户号码进行迭代，以特定的速率(-r 100参数指定，即每秒发送100次消息)，反复执行-m参数指定的次数(200000次)，并且以后台方式执行(-bg参数)。

接着调用ssh接口模块的执行命令函数执行该命令：

Ssh_module.ssh_exec_cm(SIPp_cmd)

这样，SIPp控制模块即通过ssh接口，在SIPp工具节点上启动一个测试流程。

3.2.2 在测试过程中动态调整SIPp发送消息的速率

压力测试过程中，要求周期性地变化压力程度，即SIPp发送消息的速率需要变化。在启动脚本中，我们通过-r参数指定了初始的消息发送速率，后续SIPp会按此速率持续运行。在人工测试时，我们可以在SIPp的运行界面上，键入“+”、“-”、“*”、“/”动态地调整消息发送速率。但在自动化测试过程中，没有人工干预，并且SIPp运行于后台模式(即SIPp静默运行，无人机交互界面，也没有打印输出)。因此需要另想途径调整SIPp的速率。通过研究SIPp的相关文档，发现SIPp实例在运行时，预留有UDP协议的管理端口，该端口从8888开始，每增加一个实例，该端口也顺次加1，即第一个运行的SIPp实例的管理端口是8888，第二个实例的管理端口是8889，依次类推。可以通过向该端口发送udp消息的方式，来控制SIPp实例的运行。例如要调整第一个SIPp实例的消息发送速率为每秒1000次，相应的Linux shell命令如下：

echo "cset rate 2000" >/dev/udp/127.0.0.1/8888

通过ssh接口模块向SIPp节点发送命令的python代码如下：

Ssh_cmd = ‘echo "cset rate 2000" >/dev/udp/127.0.0.1/8888’

Ssh_module.ssh_exec_cm(SIPp_cmd)

3.2.3 停止SIPp脚本

压力测试结束后，需要自动停止SIPp节点的消息发送，此处有两种方式：优雅方式和强制方式。 前者通过给SIPp发送结束命令，让SIPp缓慢地结束整个测试流程，即：不再发送新的消息，然后等待已经发出的消息全部收到响应后，将整个流程关闭；而强制方式则不管已发送的消息是否已经得到响应，直接将SIPp实例强行停止(通过linux的pkill命令)。

对于优雅模式，我们仍然使用SIPp的管理端口，相应的python代码为：

Ssh_cmd = ´echo "q" > /dev/udp/127.0.0.1/8888´

Ssh_module.ssh_exec_cm(SIPp_cmd)

在本系统实现中，先对所有的SIPp节点的所有实例执行优雅停止命令，然后进行一段时间的延时，再执行强制停止SIPp进程，相应的python代码如下：

Ssh_cmd = ´pkill -9 SIPp´

Ssh_module.ssh_exec_cm(SIPp_cmd)

3.3 数据采集及分析模块

在测试过程中，以及在测试结束后，都需要采集被测试设备运行数据，以便判断设备是否运行稳定以及是否发生故障。本系统通过TCP Server/Client模型，登录被测试设备的网管接口，执行相应的采集数据命令，得到设备运行状态数据。所有接口封装为一个类，主要接口有初始化接口，登录接口、执行网管命令接口等。

初始化接口的python代码如下：

```
def __init__(self, address, bureau, ne_user, ne_pass, timeout=60):
self.client=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
self.client.connect(address)
self.client.settimeout(timeout)
self.set_ndfne(bureau)
self.login(ne_user,ne_pass)
```

执行设备网管人机命令的接口代码如下：

```
def execute_command(self, command):
self.client.send(str(command))
result = self.client.recv(RECEIVE_BUF)
return result
```

登录设备网管的接口代码如下：

```
def login(self, username="admin", password=""):
cmd = ´SLOGIN: NAME="%s",PWD="%s";´ % (username, password)
ack = self.execute_command(cmd)
if self.Get_cmd_ack_code(ack) == 0:
print ´登录成功.´
return True
else:
return False
```

4 结束语

软件测试是软件开发的一个有机组成部分，是软件质量保证的关键环节，代表了需求、设计和编码的最终检查。本文利用SIPp工具能增加系统负荷的特性以及Python脚本语言的便捷性和高可扩展性，设计了基于Python和SIPp的自动化压力测试系统，将人工测试步骤编排为自动化测试，克服了现有测试系统的不足，极大地提高了测试效率，节约了测试成本。同时，本文提供了核心部分的代码，以期为相关人员提供参考。

参考文献：

[1] Mark Lutz. Python编程[M].邹晓,译.北京：中国电力出版社, 2015.

[2] 黄建军,李宥谋,刘婧,等.基于Python语言的自动化测试系统的设计与实现[J].现代电子技术,2017(2):39-42.

[3] 何晨,曹毅.基于SIPp的SIP协议性能测试研究与应用[J].电子技术与软件工程, 2014(15):43-45.